

Mobile Application Penetration Testing Report

Sanitized Summary

Prepared by:
Maximillian Delavega Adiwinata Frenat
(member of a five-student project team)

Program Studi Cyber Security
Binus University
2026

1. Introduction

This document summarizes the mobile penetration test that our group of five students carried out on an Android application as a university course project. The aim was to test the security of the application and find its vulnerabilities. Every vulnerability we found is rated with CVSS 4.0.

The application is a consumer app with several services in one: ride booking, food ordering, package delivery, and a wallet with transfers between users. The application is intentionally not named.

This is a shortened version of the original group report, made so it can be shared publicly. The application name, server addresses, keys, tokens, device identifiers, test accounts, screenshots, the names and student IDs of the other team members, and the step-by-step exploitation were all removed. Each finding is described by its vulnerability class, its impact and the fix we recommended.

2. Scope & Methodology

The test followed the OWASP Mobile Application Security Testing Guide (MASTG) and used two approaches. In **static analysis**, the APK was decompiled with JADX and scanned with MobSF, and we read through the resources and code looking for hard-coded secrets and security logic that runs on the client. In **dynamic analysis**, we ran the app on a rooted emulator, used Frida to instrument it, and intercepted and modified its traffic with Burp Suite to see how the server reacted to changed requests. The tools we used are listed below.

Tool	Function
JADX GUI	Static analysis of the decompiled APK source code
MobSF	Automated static analysis of the APK
Android Studio 2025.2.3.9	Emulator for running and interacting with the app
Android Debug Bridge (ADB)	Shell access to the app on the emulator
Frida	Dynamic analysis and instrumentation of the running app
Burp Suite Community Edition v2026.4.3	Intercepting and modifying the app's traffic

The emulator we used had the following specification.

Emulator	Android Studio 2025.2.3.9
Device rooted	Yes
API level	36
Android version / architecture	Android 16, x86_64

Severity labels follow the CVSS 4.0 qualitative scale: None (0.0), Low (0.1 to 3.9), Medium (4.0 to 6.9), High (7.0 to 8.9) and Critical (9.0 to 10.0). The scores are our own assessment as a student team and have not been confirmed by the vendor.

3. Executive Summary

We found three vulnerabilities and one informative item. The first is a Google Maps API key stored in the app's resources without any usage restriction, so it also works outside the app. An attacker could use it to send a large number of requests, use up the owner's quota and make the company's bill grow. The key should be restricted to the application, or the calls should be moved to the server.

The second is a Client-Side Security Control Bypass. The PIN that protects the transfer feature was only enforced in the client, by a value the user can set, so the PIN screen could be skipped. PIN validation should be done on the backend only. Related to this, the balance shown on the transfer screen was read from the URL and could be edited, but the backend rejected any transaction made against the edited value, so we recorded it as informative with no financial impact.

The third is a Security Control Bypass in the device-blocking system. It relied on a device identifier sent by the client, which could be replaced to get around the block. We recommend device attestation instead.

4. Vulnerabilities Found

The vulnerabilities we found, with their CVSS 4.0 scores, are listed below.

Name	Score	Severity	String
Unrestricted Maps API Key	6.9	Medium	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:L/SA:L
Client-Side Security Control Bypass	7.1	High	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N
Security Control Bypass	5.1	Medium	CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
Parameter Tampering (informative)	0.0	None	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

The table below pairs each finding with the weakness class it falls under and the main fix we recommended in the report.

Finding	Weakness class	Main fix
Unrestricted Maps API Key	CWE-798: Use of Hard-coded Credentials	Restrict the key to the app and needed APIs, or call the service from the server
Client-Side Security Control Bypass	CWE-602: Client-Side Enforcement of Server-Side Security	Verify the PIN on the backend and keep that state in a server-side session
Security Control Bypass	CWE-807: Reliance on Untrusted Inputs in a Security Decision	Device attestation instead of a client-supplied identifier
Parameter Tampering (informative)	CWE-472: External Control of Assumed-Immutable Web Parameter	Do not pass user data through URL parameters; load it on the backend

5. Details of Findings

This section describes each finding. Component names are generalized, and exploit steps are left out.

A. Unrestricted Maps API Key

Score: 6.9 (Medium)

String: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:L/SA:L

Weakness: CWE-798

Component: application string resources

Details

Static analysis of the decompiled app showed an API key for a maps service stored in plain text in a resource file. To find out whether the key was restricted, we sent a single geocoding request to the provider's public API using only that key, from outside the app. The provider returned valid data, which shows that nothing tied the key to the application. The CVSS vector reflects this: the key is reachable over the network with no privileges or user interaction, and the effect is on the owner's other resources (quota and billing) rather than on the app itself.

Impact

An unrestricted key can lead to billing abuse. An attacker can use it to send many fake requests, use up the owner's API quota and make the company's charges pile up, which causes financial loss.

Recommendation

Do not hard-code keys that give access to a paid service, because anyone who unpacks the APK can read them. If a client-side key cannot be avoided, restrict it to the application (for example by package name and signing certificate) and to the specific APIs it needs, and set quotas and billing alerts. A key that has already been exposed this way should be rotated. Otherwise, make the calls from the server.

B. Client-Side Security Control Bypass

Score: 7.1 (High)

String: CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

Weakness: CWE-602

Component: balance transfer page

Details

The transfer feature asks for a PIN before it can be used. The PIN screen was implemented as an overlay on top of the page, and whether the overlay was shown depended on a flag kept in the browser's session storage. The transfer page and its logic had already been delivered to the client, so the overlay could be removed by setting that flag, and this gave access to the transfer screen without entering a PIN. In other words, the PIN was never enforced by the server. The vector shows low privileges required (a logged-in account) and a high impact on integrity.

Impact

A PIN that can be skipped lets an attacker transfer balance to another account easily. An attacker who has taken over a user's account could move the user's money without knowing the PIN.

Recommendation

1. The page should return only the PIN form until the PIN has been verified, and should not ship the protected screen behind an overlay.
2. After a correct PIN is submitted, the backend should set a secure server-side session state, instead of relying on client-side storage.
3. PIN validation should be done entirely on the backend. Limiting the number of PIN attempts on the server is also advisable.

Informative: Parameter Tampering

Score: 0.0 (None)

String: CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

Weakness: CWE-472

Component: balance transfer page

Details

The balance shown on the transfer screen was passed to the page as a URL query parameter, and changing it changed the balance displayed. This shows that the app trusted a client-supplied value for display purposes. However, when we attempted a transfer against the altered balance, it failed, because the backend compared the amount with the real balance stored in its database. This is why the finding scores 0.0: the display could be spoofed, but money could not be moved.

Recommendation

Transactions and user data should be handled entirely by the backend instead of being sent through the URL from the client. Sensitive values such as balance or name should not be passed as GET parameters, because they can be abused for visual spoofing or social engineering.

C. Security Control Bypass

Score: 5.1 (Medium)

String: CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

Weakness: CWE-807

Component: device check at registration and login

Details

While testing, the emulator we were using was blocked from the app. Registering and logging in both failed, and the app showed a message saying access was refused. When we intercepted the registration request, we saw that the app sends a device identifier (IMEI) to the server, which uses it to recognize banned devices. After we changed that identifier in the intercepted request, the server accepted the registration, and we could use the app again to continue testing. The block therefore depended on a value chosen by the client, and a blocked device could present a new identity whenever it wanted.

Impact

An attacker can fake a device identity to get past the device-blocking system and create accounts without authorization.

Recommendation

Replace the client-side validation with device attestation, for example the Play Integrity API on Android, so that the server decides based on a signal the client cannot simply edit.

6. Conclusion

The penetration test of this application found vulnerabilities of different severity levels. They share one root cause: the client was trusted to make security decisions. The PIN gate, the device block and the balance shown on screen all depended on data that the user can read and change, and the API key was available to anyone who unpacked the APK. The one check that held up was the transaction validation, which compared the values against the database on the server.

The recommended fixes follow the same principle. Validation and authorization should be done by the backend, secrets should not ship inside the app, and device attestation should replace identifiers supplied by the client.

These findings describe the application at the time of our testing. They may have changed since then, and this summary should be read as a record of a course project, not as a current assessment of the product.